LAB 1 · LECTURE 1

Lecture 1 Lab: Keyword vs Semantic Search

Vector Databases Course | Estimated time: 30-40 minutes | Platform: Google Colab (free)

Goal

In this lab you will build two tiny search engines over the same small dataset: one that matches **words** and one that matches **meaning**. You will then compare them and find queries that break the keyword approach.

By the end you will be able to:

- Implement a simple keyword search in plain Python.
- Turn sentences into embeddings with the sentence-transformers library.
- Rank documents by cosine similarity (a preview of Lecture 3).
- Explain, with examples, why semantic search finds results keyword search misses.

Setup

1. Open **colab.research.google.com** and create a new notebook.
2. Paste each code block below into its own cell and run the cells in order.
3. No GPU is needed. The first run downloads a small model (about 90 MB).

Cell 1: install and import

```
!pip install -q sentence-transformers numpy pandas

import re
import numpy as np
import pandas as pd
from sentence_transformers import SentenceTransformer
```

Part 1: The dataset

We use 12 short travel and tech help-center sentences. Notice that several pairs say the same thing in different words.

Cell 2: documents

```
docs = [
    "Affordable hotels near the beach",
    "Cheap flights to Cairo",
    "Budget hostel for backpackers",
    "How to reset your account password",
    "I forgot my login credentials",
    "Best restaurants for seafood lovers",
    "Where to eat fresh fish by the sea",
    "Laptop battery drains too quickly",
    "My computer's charge does not last long",
    "Guided tours of the pyramids",
    "Book a sightseeing trip to ancient monuments",
    "Refund policy for cancelled bookings",
]
print(len(docs), "documents loaded")
```

Part 2: Keyword search from scratch

Our keyword engine splits text into lowercase words and scores each document by how many query words it contains. This is a simplified version of what SQL `LIKE` or an inverted index does.

Cell 3: keyword search

```
def tokenize(text):
    return set(re.findall(r"[a-z0-9']+", text.lower()))

def keyword_search(query, docs, top_k=3):
    q = tokenize(query)
    scored = []
    for i, d in enumerate(docs):
        score = len(q & tokenize(d))   # number of shared words
        scored.append((score, i))
    scored.sort(reverse=True)
    return [(docs[i], score) for score, i in scored[:top_k]]

def show(results):
    for text, score in results:
        print(f"  {score:.3f}  {text}")

query = "cheap places to stay"
print("KEYWORD results for:", query)
show(keyword_search(query, docs))
```

Expected behavior: only documents that literally contain a query word (such as "cheap" or "to") get a non-zero score. The beach hotel and the hostel, which are the best answers, score 0.

YOUR TURN Run keyword search for the queries "I lost my password" and "places to eat seafood". Write down which relevant documents are missed.

Part 3: Semantic search with embeddings

Now we replace word matching with **embeddings**. The model `all-MiniLM-L6-v2` converts each sentence into a vector of 384 numbers.

Cell 4: embed the documents

```
model = SentenceTransformer("all-MiniLM-L6-v2")

doc_vectors = model.encode(docs, normalize_embeddings=True)
print("Shape:", doc_vectors.shape)       # (12, 384)
print("First 5 numbers of doc 0:", doc_vectors[0][:5])
```

Because we pass `normalize_embeddings=True`, every vector has length 1. For unit vectors, cosine similarity equals the dot product, which keeps the code short.

Cell 5: semantic search

```
def semantic_search(query, docs, doc_vectors, top_k=3):
    q_vec = model.encode([query], normalize_embeddings=True)[0]
    scores = doc_vectors @ q_vec            # cosine similarity per document
    best = np.argsort(-scores)[:top_k]      # indices of highest scores
    return [(docs[i], float(scores[i])) for i in best]

query = "cheap places to stay"
print("SEMANTIC results for:", query)
show(semantic_search(query, docs, doc_vectors))
```

Expected behavior: the hotel and the hostel now rank at the top even though they share no words with the query. Exact scores may differ slightly between library versions.

Part 4: Side-by-side comparison

Cell 6: compare both engines on several queries

```
queries = [
    "cheap places to stay",
    "I lost my password",
    "places to eat seafood",
    "my laptop dies fast",
    "visit ancient sites",
    "get my money back",
]

rows = []
for q in queries:
    kw  = keyword_search(q, docs, top_k=1)[0]
    sem = semantic_search(q, docs, doc_vectors, top_k=1)[0]
    rows.append({
        "query": q,
        "keyword top-1": kw[0] if kw[1] > 0 else "(no match)",
        "semantic top-1": sem[0],
        "semantic score": round(sem[1], 2),
    })

pd.set_option("display.max_colwidth", 50)
pd.DataFrame(rows)
```

YOUR TURN Fill in this table from your output. Mark each result as Good, Partly, or Bad.

Query	Keyword top-1	Good?	Semantic top-1	Good?
cheap places to stay				
I lost my password				
places to eat seafood				
my laptop dies fast				
visit ancient sites				
get my money back				

Part 5: Break it

Semantic search is powerful but not magic. Explore its limits:

- Write **three queries** where keyword search clearly fails but semantic search succeeds.
- Write **one query** where semantic search returns something wrong or surprising. Hint: try a very short query, a product code like "A-4471", or an exact name.
- Add 3 new documents of your own to `docs`, then re-run Cells 4 to 6.

Cell 7 (optional): inspect a similarity score

```
a, b = model.encode(["I forgot my login credentials",
                     "How to reset your account password"], normalize_embeddings=True)
c = model.encode(["Guided tours of the pyramids"], normalize_embeddings=True)[0]

print("related   :", float(a @ b))
print("unrelated :", float(a @ c))
```

Part 6: Reflection questions

- Why did the keyword engine return "Cheap flights to Cairo" for a hotel query?
- What would happen if you embedded the documents with one model and the query with another? Why?
- Keyword search is still better for some tasks. Name two (hint: exact identifiers, legal phrases).
- Our search compares the query with all 12 documents. What would go wrong with 100 million documents? (This leads to Lecture 4.)

Deliverable

Submit your notebook with all cells executed, the completed comparison table, and short answers to the four reflection questions.

Troubleshooting

- **ModuleNotFoundError:** re-run Cell 1 and restart the runtime if Colab asks.
- **Model download is slow:** it happens only on the first run; wait for it to finish.
- **Different scores than the handout:** small variations between library versions are normal. The ranking should look similar.